Market Simulation for Algorithmic Trading

by

BHANDARI Om Raj
CHOY Ping Hang Gordon
KWONG Wing Fung Josh

Advised by

Prof. Pedro SANDER

Submitted in partial fulfillment of the requirements for COMP 4981
(PSAN4)

in the

Department of Computer Science

The Hong Kong University of Science and Technology

2025-2026

Contents

1 Introduction **4**

1.1 Overview . . . 4

1.1.1 Introduction to Algorithmic Trading . . . 4

1.1.2 Need for Trading Simulation . . . 4

1.1.3 Project Contributions . . . 4

1.2 Objective . . . 5

1.3 Literature Survey . . . 6

1.3.1 Limit Order Book architecture . . . 6

1.3.2 System-level architecture . . . 6

1.3.3 Simulation methodologies . . . 7

2 Methodology **8**

2.1 Design . . . 8

2.1.1 User Access Layer: SDK & Graphical Interface . . . 8

2.1.2 External Facing API Layer: FIX Gateways . . . 9

2.1.3 Order Validation & Persistence Layer: Order Manager . . . 9

2.1.4 The Core: MatchingEngine & LOB Data Structures . . . 9

2.1.5 Distribution Layer: Market Data Processor . . . 10

2.1.6 Simulation Layer: Agents & Market Regimes . . . 10

2.2 Implementation . . . 11

2.2.1 User Access Layer: SDK & Graphical Interface . . . 12

2.2.2 External Facing API Layer: FIX Gateways . . . 14

2.2.3 Order Validation & Persistence Layer: Order Manager . . . 15

2.2.4 The Core: MatchingEngine & LOB Data Structures . . . 16

2.2.5 Distribution Layer: Market Data Processor . . . 18

2.2.6 Simulation Layer: Agents & Market Regimes . . . 18

2.2.7 Server Setup and Deployment . . . 19

2.2.8 Software Engineering Practices . . . 20

2.3 Testing . . . 21

2.3.1 Test Plan and Methodology . . . 21

2.3.2 Locating Logic Errors and Bugs . . . 21

2.4 Evaluation . . . 22

2.4.1 Benchmarking . . . 22

2.4.2 Simulation Realism . . . 26

3 Discussion **27**

3.1 Project Evaluation and Reflections . . . 27

3.2 System Limitations 27

4 Conclusions **29**

5 References **30**

6 Appendix **31**

6.1 Explanation on Limit Order Books 31

6.2 Project Planning 32

6.2.1 Distribution of Work & Gantt Chart 32

6.3 Required hardware & software 32

6.3.1 Hardware 32

6.3.2 Software 32

6.4 Comprehensive Test Cases 33

Abstract

1 Introduction

1.1 Overview

1.1.1 Introduction to Algorithmic Trading

Financial markets are complex, dynamic systems where algorithmic trading plays a crucial role in price discovery, liquidity provision, and market efficiency. Hong Kong, as one of the world's leading financial hubs, attracts top institutions and professionals who leverage advanced trading algorithms to remain competitive, including investment banks, hedge funds, and market makers. However, there is minimal access to high-quality, open-source algorithmic trading platforms that allow newcomers, such as university students, to interact with Limit Order Books.

1.1.2 Need for Trading Simulation

We were inspired by algorithmic trading competitions such as IMC Prosperity that demonstrate the educational value of simulated markets. Participants design systematic strategies and deploy them against unseen market data containing IMC-proprietary bots that are configured with diverse behaviours, such as market making. Unfortunately, the code is closed-source and inaccessible for independent learning. This project seeks to fill the gap by creating an educational trading simulator.

1.1.3 Project Contributions

This project contributes an extensible algorithmic trading simulator and a systematic empirical study of Limit Order Book implementations, with particular emphasis on data structure design and low-latency performance characteristics in high-volume settings. The simulator recreates the environment of real-world trading by supporting multiple assets, heterogeneous agent populations, and configurable market regimes, thereby providing a controlled testbed for both strategy evaluation and microstructure experimentation.

The primary technical contribution lies in successfully "white-boxing" the traditionally opaque and highly complex architecture of public financial exchanges. While real-world exchanges operate as proprietary black boxes, this project scales these massive systems down into an accessible educational format—a process of systematic architectural down-scaling. By synthesizing industry research and structural estimations, we engineered a transparent, functional system. Specifically, we contribute the design, implementation, and comparative evaluation of several financial exchange backends that realize the same matching logic over different underlying data structures, such as node-based containers (e.g. balanced binary search tree-style price maps with per-level linked queues) versus

contiguously stored containers (e.g. vector-like arrays of price levels with in-place order queues). This enables a direct comparison of how memory layout, pointer indirection, and CPU cache locality affect core operations such as insertion, cancellation, best-quote retrieval, and full-book traversal. We also experimented with and implemented modular architectures which utilize concurrency techniques such as lock-free ring buffers.

The secondary educational contribution is the parameterizable multi-scenario configurations that expose microstructural variables to the user, such as agent order-submission intensities, volatility and regime dynamics, tick size, and artificial latency. These scenarios support different classes of agents (e.g. market makers, noise traders, momentum and value strategies) and can be instantiated per symbol and per "trading day". Alongside the simulations, the project contributes a visual analytics layer that presents a time series of prices, spreads, depths, and selected microstructural indicators derived from the simulated LOB state. The simulation and visualizations make the platform suitable for instructional use in courses on market microstructure and algorithmic trading.

1.2 Objective

The goal of this project is to provide a safe and realistic venue for beginners in algorithmic trading to learn and test their skills. To facilitate this, we designed and implemented a robust simulated market platform for real-time trading. This platform allows users to test their trading strategies in realistic market scenarios and refine their skills through trial and error. Therefore, our objectives revolve around realistic market simulation and the ability to handle large volumes of trades, mimicking a real-world stock exchange.

The main objectives of the project were as follows:

1. To build an efficient Limit Order Book that could handle incoming orders at the lowest latency possible.
2. To provide an intuitive method for users to submit orders.
3. To provide a clear and clean interface for visualising market information.
4. To design an efficient and robust system architecture that can handle high user traffic.
5. To implement realistic and configurable market simulation mechanisms.

For objectives 1 and 4, we conducted research on existing Limit Order Book designs, stock exchange architecture diagrams, and C++ conferences (cppcon). The researched results were then benchmarked to arrive at the most efficient solutions. To achieve objective 2, we designed and implemented a Software Development Kit (SDK) strictly in C++ to guarantee high performance and minimal latency overhead. To satisfy objec-

tive 3, we studied interfaces of commonly used brokerage applications, including IBKR and Webull. Our study allowed us to pinpoint necessary information to display and understand how data should be presented in a clear manner. For objective 4, we built a distributed architecture that can support a high volume of user-submitted orders. Finally, for objective 5, we reviewed research papers proposing theoretically plausible methods of market simulation, narrowing them down to several methods that we closely tested and monitored.

1.3 Literature Survey

This literature survey examines existing work on the three major components of a financial exchange: the Limit Order Book, system-level architecture, and simulation methodologies.

1.3.1 Limit Order Book architecture

The Limit Order Book sits at the core of an exchange. In real exchanges, an individual Limit Order Book for a single asset must handle message rates up to 500,000 per second with microsecond latencies [3]. Correct and efficient implementation is crucial. Essentially, it must implement add, cancel, and execute operations for orders, with efficient queries such as getting the best bid and ask prices, current volume information, and the status of the order submitted by a specific user. For a detailed explanation, refer to Appendix 6.1. Current designs utilise binary trees as the primary data structure to store orders, with each node referring to a limit price. Each node then contains a doubly linked list of order nodes corresponding to those that users submit. To solve the issue of efficient queries, two hashmaps store references to limit price nodes of the tree and order nodes in the linked list, respectively [5]. However, current designs do not explicitly define which type of binary tree to use for keeping the trees balanced, as the average time complexity may grow if left unmaintained. Furthermore, the literature typically only considers theoretical time complexity and lacks empirical performance metrics, such as benchmarked latency and its distributions. It also frequently ignores computer architecture considerations, such as cache locality and cache line optimization.

1.3.2 System-level architecture

Modern exchanges handle thousands of assets, each with its own Limit Order Book, and multiple concurrent users submitting orders. Across all traded assets, an exchange can face an aggregate throughput upwards of 3,000,000 messages per second at peak levels [3]. Most exchanges do not reveal their source code due to competitive advantages. Hence, existing literature only describes the architecture of exchanges at a high level [1]. Users first send messages regarding their order intentions to an exchange gateway layer, which

performs validation on message integrity and runs on a separate computer from the core Limit Order Books. This layer contains multiple parallel gateways for users to choose from and send messages to. To handle simultaneous messages and introduce a level of randomness to message handling, a sequencer first materialises the incoming messages into a single stream of ordered messages. This stream is then passed on to the Limit Order Books for order matching, addition, or cancellation. However, current literature does not concretely define the implementation details of this architecture, such as the message and network connection protocols, load-balancing mechanisms, and inter-service communication.

1.3.3 Simulation methodologies

It is often useful to include order simulation to model behaviours of market participants. Existing open-source exchange simulation tools (such as LOBSTER or ABIDES) provide historical market data replay mechanisms and simple random order generation to model noise. However, these simulators often fall short for interactive educational purposes: they are typically implemented in slower, high-level languages like Python rather than low-latency C++, and they critically lack a real-time graphical user interface for visualizing market depth and live trading [4].

By contrast, quantitative finance literature provides rigorous mathematical frameworks for Limit Order Book simulation. For instance, the Avellaneda-Stoikov model formally describes optimal quoting strategies for market makers managing inventory risk, while Geometric Brownian Motion (GBM) is a standard continuous-time stochastic process used to model fundamental asset price trajectories [2]. While these models provide strong theoretical foundations, the literature does not provide the robust source code necessary to implement such processes in a live, high-frequency stock exchange program.

The surveyed literature and existing platforms demonstrated useful designs for the core components of our exchange project. However, most works remained either too high-level in terms of implementation or lacked the performance and user-facing interactivity required for a modern sandbox. Our project aimed to integrate all three of these component designs to implement an efficient, C++-based, and scalable platform that is highly educational, visual, and supports advanced mathematical simulation methods.

2 Methodology

2.1 Design

The system architecture of the simulated exchange is engineered as a distributed, modular pipeline, closely mirroring the microstructure of real-world financial exchanges. It is logically partitioned into distinct microservices: user access, message validation, state persistence, core order matching, and market data distribution. The data flow follows a strict, unidirectional path. User order requests enter through the External Facing API Layer, undergo semantic validation and database logging in the Order Manager, and are synchronously routed to the MatchingEngine. The engine executes the order matching algorithms and outputs raw trade events. These events are immediately consumed by the Market Data Processor via a shared-memory ring buffer, transformed into aggregated market data, and broadcast to connected clients. This decoupled architecture intentionally isolates the latency-critical matching core from network and database I/O overhead, enabling horizontal scalability under high-volume algorithmic trading scenarios.

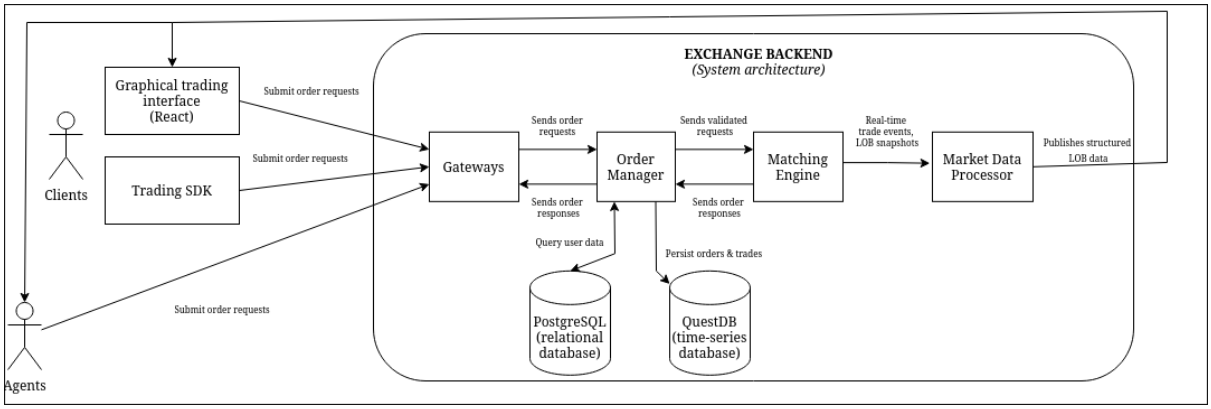


Figure 1: High-level system architecture of the exchange backend.

2.1.1 User Access Layer: SDK & Graphical Interface

To provide a seamless experience for both developers and retail-like users, the system exposes two primary interaction points:

- **Trading SDK:** We developed a programmatic interface that abstracts complex networking logic into simple, intuitive commands for order submission and cancellation. This SDK acts as a high-performance bridge between algorithmic strategies and the trading system. To handle real-time execution reports (automated system messages confirming whether an order has been successfully placed, matched, or canceled), the SDK provides an event-driven architecture where users can define custom callback handlers to react to market updates seamlessly.

- **Graphical Trading Interface:** We designed a web-based dashboard inspired by professional brokerages. It serves as the visual analytics layer, rendering real-time market depth and price action. Crucially, this interface also allows users to instantiate their own customized, isolated trading environments. Analogous to "game servers" in multiplayer gaming, each user can create a server that hosts an isolated market with its own distinct set of tickers. This provisioning is orchestrated by the frontend communicating with a backend management service and a deployment manager, ensuring complete sandbox isolation for algorithmic experimentation.

2.1.2 External Facing API Layer: FIX Gateways

Mirroring real-world exchange architecture, all incoming user requests first hit the gateways to undergo simple message validation before being parsed into internal representations. This layer acts as a protocol translator, handling session management (logins/logouts) and ensuring that incoming requests are syntactically correct. By decoupling connection handling from the core matching logic, the system can scale horizontally to handle high user traffic. For instance, additional gateway instances can be dynamically spawned across different machines when demand spikes, without slowing down the core matching engine.

2.1.3 Order Validation & Persistence Layer: Order Manager

Once a request passes the gateway, it moves to the Order Manager, which serves as the central hub of the backend. It is responsible for semantic validation and data persistence:

- **Semantic Validation:** The manager performs critical checks that the gateway cannot, such as balance checking (verifying sufficient funds for a buy order) and position tracking (verifying sufficient inventory for a sell order).
- **Database Integration:** It interfaces with two distinct databases: a relational database for user profiles, credentials, and account balances; and a time-series database for high-frequency order data. Every order and trade event is persisted to ensure a robust audit trail and to allow for historical simulation replays.

After validation, the manager synchronously routes requests to the matching engine and subsequently passes trade confirmations back to the gateway to notify the originating user.

2.1.4 The Core: MatchingEngine & LOB Data Structures

This layer is the performance-critical heart of the exchange. The engine receives a validated request, attempts to match it against the opposite side of the Limit Order Book (LOB), generates trade events if a match occurs, and updates the LOB state.

- **Matching Logic:** We utilized the Price-Time Priority algorithm, meaning orders with the best price are matched first, whereas orders at the same price level are executed on a first-come-first-served basis.
- **Data Structure Design:** We designed a theoretically optimal LOB based on algorithmic time complexity. The bid and ask sides are represented as balanced Binary Search Trees (BSTs) to provide $\mathcal{O}(\log N)$ access to price levels. Each price level contains a doubly-linked list of orders to support $\mathcal{O}(1)$ additions at the back of the queue, preserving Price-Time Priority. This is complemented by a hash map that stores iterators to every active order, enabling $\mathcal{O}(1)$ lookups and arbitrary order cancellations without traversing the book.

2.1.5 Distribution Layer: Market Data Processor

The output from the MatchingEngine is raw and high-velocity. The Market Data Processor acts as the system's publisher, transforming core trade events and orderbook updates into accessible formats and broadcasting them to external clients.

To maintain the performance gains achieved by the core engine, the inter-process communication (IPC) channel between the engine and the processor relies on a shared memory mechanism. This architectural choice guarantees that the matching engine is completely decoupled from the unpredictable latency introduced by string serialization and network broadcasts. By strictly delegating these computationally intensive I/O tasks to a separate process, the core engine can continue matching orders unimpeded.

2.1.6 Simulation Layer: Agents & Market Regimes

To provide a realistic and educational venue, the system includes an agent layer that simulates heterogeneous market participants interacting with the gateways just like human clients. We designed three distinct agent-based models to represent different market microstructure participants:

- **Noise Traders:** Operating as a zero-intelligence model, these agents simulate uninformed retail flow. Their order arrivals are governed by a Poisson process, meaning the inter-arrival time Δt between consecutive orders follows an exponential distribution:

  $$f(\Delta t; \lambda) = \lambda e^{-\lambda \Delta t} \tag{1}$$

  where λ represents the order intensity (events per second). Order parameters (side, quantity, limit vs. market) are generated using randomized statistical distributions to provide baseline market activity.
- **Market Makers:** Implementing the Avellaneda-Stoikov model, these agents pro-

vide liquidity by maintaining a ladder of quotes around the spread. They actively manage inventory risk by calculating a reservation price r (the theoretical mid-price adjusted for inventory) and an optimal spread δ:

$$r = s - q\gamma\sigma^2(T - t) \tag{2}$$

$$\delta = \gamma\sigma^2(T - t) + \frac{2}{\gamma}\ln\left(1 + \frac{\gamma}{k}\right) \tag{3}$$

Here, s is the current market mid-price, q is the agent's inventory, γ is the risk aversion parameter, σ^2 is the asset's price variance, $T - t$ is the time horizon, and k represents the order book density. The optimal bid and ask quotes are then symmetrically placed at $r - \delta/2$ and $r + \delta/2$.

- **Informed Traders & Oracle Service:** These agents act on a "true" fundamental asset price V_t provided by an external Oracle Service. The oracle generates this true price path using a Geometric Brownian Motion (GBM) augmented with Jump Diffusion to simulate sudden exogenous shocks:

$$dV_t = \mu V_t dt + \sigma V_t dW_t + V_t dJ_t \tag{4}$$

  where μ is the drift, σ is the volatility, W_t is a standard Wiener process, and J_t represents the jump process. The Informed Trader continuously polls the order book and the oracle. If it detects a mispricing greater than a threshold ϵ (e.g., $V_t > P_{ask} + \epsilon$), it executes aggressive snipe orders against lagging market maker quotes. This introduces adverse selection and efficiently drives the simulated market towards price discovery.

For the purpose of this educational sandbox, the mathematical parameters governing these agents (e.g., the risk aversion γ, drift μ, and volatility σ) are initialized with arbitrary defaults rather than being strictly calibrated to historical market data. However, they are fully configurable via the frontend interface and deployment configuration files, allowing users to experiment with diverse market regimes such as high-volatility environments or directional trend markets.

2.2 Implementation

Our implementation leverages modern C++23, prioritizing low-latency performance and robust type safety. The project uses CMake as its primary build system, facilitating dependency management and modular compilation of the various microservices. External dependencies such as QuickFIX, Boost, Protocol Buffers, and various database clients are integrated directly into the CMake workflow.

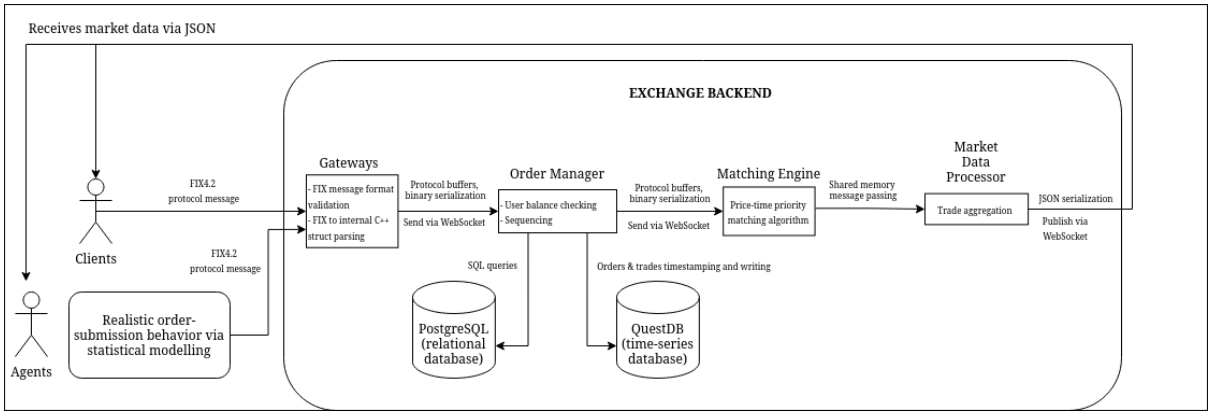


Figure 2: Detailed technical implementation of the exchange backend components.

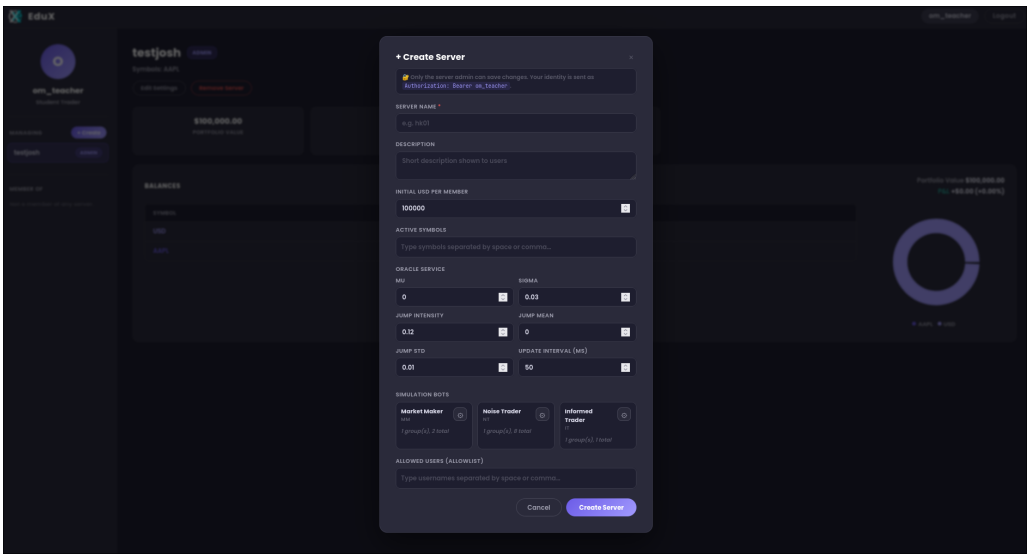
To facilitate robust and efficient network communication, we developed a dedicated `transport` library. For structured message serialization, we utilize Google's Protocol Buffers strictly for internal, inter-service communication (such as routing validated orders from the gateway to the order manager). Their compact binary representation and rapid serialization speeds make them ideal for maintaining a low-latency trading pipeline, whereas text-based formats like JSON are reserved primarily for external communication with the frontend and public APIs. Furthermore, we implemented a custom internal WebSocket library wrapping `websocketpp`. For architectural convenience, we standardized on WebSockets for almost all internal inter-service communication (with the notable exception of the MatchingEngine to Market Data Processor link, which uses a shared-memory ring buffer for ultra-low latency). This same WebSocket infrastructure is seamlessly reused to disseminate real-time market data feeds to the public React frontend.

2.2.1 User Access Layer: SDK & Graphical Interface

The client SDK was developed entirely in C++ to provide developers with a fast and intuitive interface for connecting to the trading system. It exposes a `FixClient` class that developers can subclass to interact with the market. Key methods include `submit_limit_order`, `submit_market_order`, and `cancel_order`, alongside connection management functions. To handle asynchronous execution reports and cancel rejections, the SDK provides virtual callback methods (`on_order_update` and `on_order_cancel_rejected`) that users can override to implement custom event-driven trading logic.

For the graphical trading interface, we built a responsive single-page application (SPA) using React (`frontend-v2/src`). The frontend consumes the real-time WebSocket streams from the backend and leverages charting libraries such as `lightweight-charts` and `recharts` to dynamically render interactive candlestick charts, price action, and order book depth. Furthermore, the frontend provides a control interface for users to cre-

ate and manage their isolated market servers. This functionality is achieved by communicating with a backend REST service (`services/backend_service`) that orchestrates the provisioning of new environments via a centralized Python deployment script (`deployment_server.py`), allowing each user to effortlessly spin up their own independent market "game server."



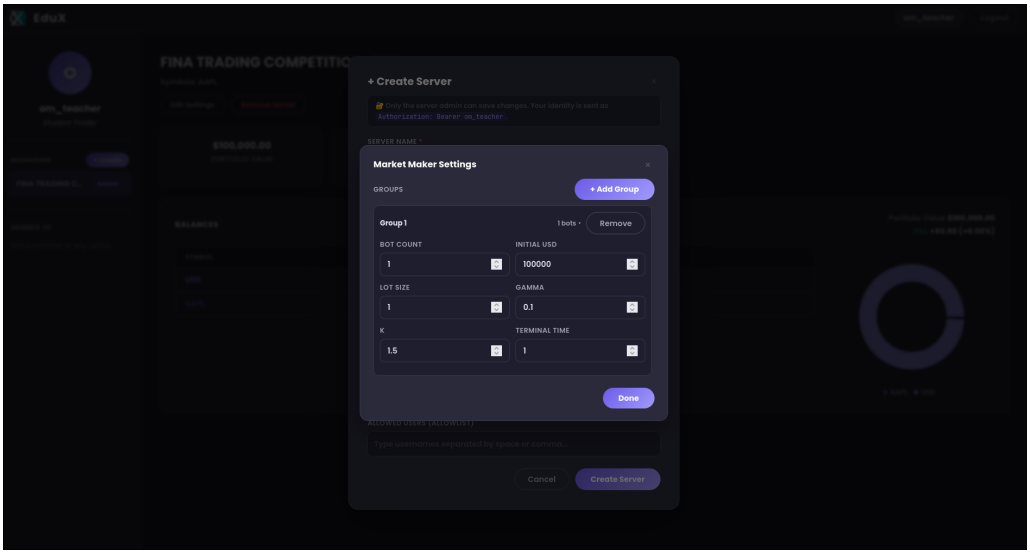


Figure 3: Server configuration and deployment interface.

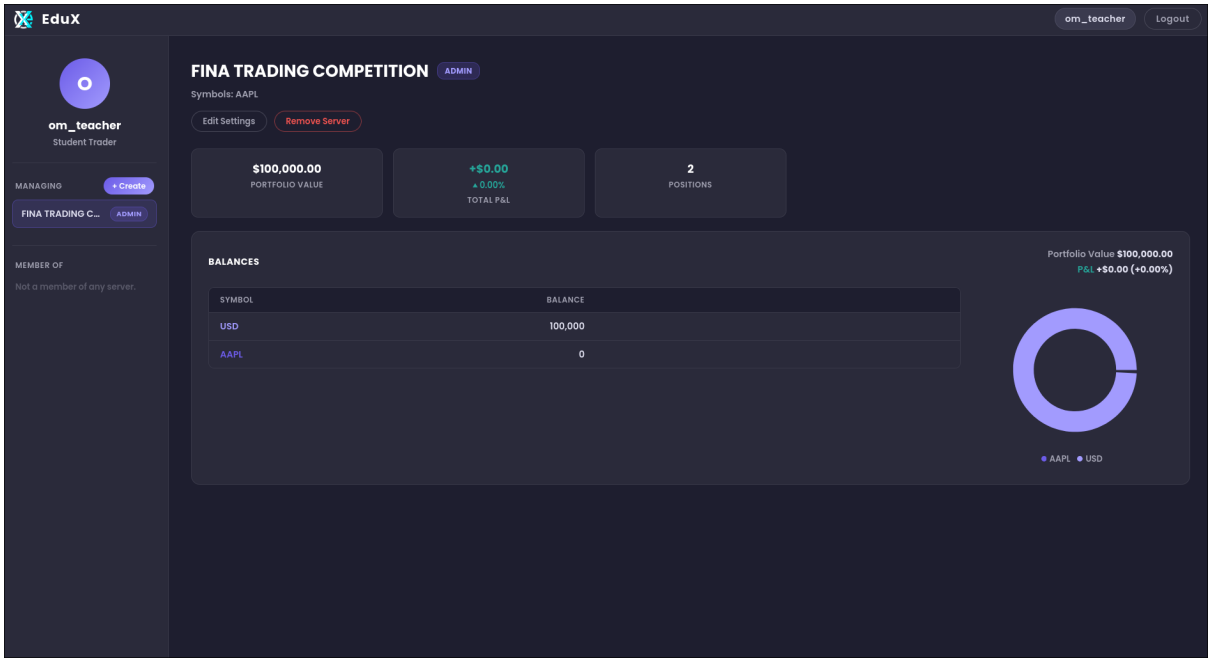


Figure 4: User portfolio and order management dashboard.

Figure 5: Real-time K-line charting and market depth visualization.

2.2.2 External Facing API Layer: FIX Gateways

The FIX Gateway was implemented using the QuickFIX C++ engine. It serves as the external-facing API, parsing incoming FIX protocol messages and translating them into

internal data structures. By leveraging QuickFIX, the gateway natively handles session management, message sequencing, and heartbeat mechanisms, ensuring reliable communication with connected clients.

2.2.3 Order Validation & Persistence Layer: Order Manager

The Order Manager acts as a synchronous validator and router. To eliminate race conditions—such as a single user submitting two massive orders simultaneously—the manager processes incoming messages sequentially within a single-threaded event loop. This guarantees that balance constraints and position limits are checked and updated atomically before any request is forwarded to the MatchingEngine. To ensure durability without blocking this critical trading path, the persistence layer utilizes a dual-database architecture managed by a custom `DatabaseClient`.

Relational data, such as account states and user configurations, is managed using a PostgreSQL database accessed via the `libpqxx` client. Conversely, high-throughput market events and tick data are logged to a QuestDB time-series database. To prevent database I/O from bottlenecking the system, we implemented an `AsyncWriter` for QuestDB ingestion. This module offloads database writes to a dedicated background thread, buffering operations like order insertions and trade logs into a thread-safe queue. The background thread batches these tasks and flushes them to QuestDB over the Ingress Line Protocol based on configurable thresholds and intervals. While validation and matching engine forwarding are synchronous to ensure correctness, this asynchronous persistence guarantees that database latency does not degrade order processing times.

To illustrate the persistence schema, the time-series definitions for order logs and trade logs in QuestDB are summarized in Tables 1 and 2. They are partitioned by day and deduplicated using UPSERT keys, which allows robust, high-throughput historical reconstruction.

Field Name	Data Type
ts	TIMESTAMP
order_id	INT
cl_order_id	INT
sender_comp_id	VARCHAR
symbol	SYMBOL
side	SYMBOL
order_qty	INT
filled_qty	INT
ord_type	SYMBOL
price	INT
time_in_force	SYMBOL
order_status	SYMBOL

Table 1: QuestDB Schema for the Orders Table.

Field Name	Data Type
ts	TIMESTAMP
trade_id	VARCHAR
price	INT
quantity	INT
symbol	SYMBOL
taker_id	VARCHAR
maker_id	VARCHAR
taker_order_id	INT
maker_order_id	INT
is_taker_buyer	BOOLEAN

Table 2: QuestDB Schema for the Trades Table.

2.2.4 The Core: MatchingEngine & LOB Data Structures

At the core of the system is the MatchingEngine, which manages the Limit Order Book (LOB). The LOB was implemented using C++ standard library node-based containers to achieve the theoretically optimal time complexities. Specifically, `std::map` maintains the sorted price levels (acting as the BST), `std::list` maintains the doubly-linked queue of orders at each price level, and `std::unordered_map` provides constant-time lookups to the list iterators for instant order cancellations. To communicate executions and market state changes to downstream services, the matching engine writes to a custom Multiple-Producer Single-Consumer (MPSC) lock-free ring buffer. This ring buffer is backed by POSIX shared memory (`shm_open` and `mmap`).

To achieve optimal IPC throughput, the buffer capacity was constrained to a power of two, replacing expensive modulo operations with bitwise AND operations for index wrapping. Additionally, to prevent false sharing, the internal structures incorporate explicit memory alignment and padding (`alignas(64)` for x86 architectures), ensuring producer and consumer indices reside on separate CPU cache lines. In the edge case where the Market Data Processor (Consumer) falls behind the MatchingEngine (Producer), the lock-free ring buffer will report as full. Currently, the engine utilizes a non-blocking `try_push` mechanism (as seen in the `try_publish` calls); if the buffer is at capacity, new events are dropped to prevent the core matching logic from blocking, emphasizing the

need for appropriately sized buffers and fast consumer polling.

Listing 1 demonstrates the critical path logic for crossing an incoming order against resting liquidity. The algorithm sequentially walks the best available price levels, decrementing quantities and generating `Trade` events until the incoming order is either fully filled or the opposing liquidity at marketable prices is exhausted.

```
void LimitOrderBook::match_order(SideContainer& near_side, SideContainer& far_side, int price,
                                 int remaining_quantity, int order_id, Side side,
                                 std::string_view broker_id) {
    // 1. Iterate through the opposing side of the book as long as we have quantity to fill
    while (!far_side.empty() && remaining_quantity > 0) {
        const auto best_level = (side == Side::bid) ? far_side.begin() : std::prev(far_side.end());
        const auto& best_level_price = best_level->first;

        // 2. Stop matching if the best available price is no longer marketable
        if ((side == Side::bid && price < best_level_price) ||
            (side == Side::ask && price > best_level_price)) {
            break;
        }

        auto& best_level_orders = best_level->second;
        // 3. Execute against orders at the current best price level (Price-Time Priority)
        while (remaining_quantity > 0 && !best_level_orders.empty()) {
            auto& front_order = best_level_orders.front();

            if (const auto order_quantity = front_order.get_quantity();
                remaining_quantity >= order_quantity) {
                // 3a. Incoming order fully consumes the resting order
                remaining_quantity -= order_quantity;
                Trade new_trade = create_trade(order_id, front_order.get_order_id(), broker_id,
                                               front_order.get_trader_id(), ticker,
                                               front_order.get_price(), order_quantity, side);
                trade_publisher->try_publish(new_trade);
                trade_events.emplace(std::move(new_trade));
                order_id_map.erase(best_level_orders.front().get_order_id());
                best_level_orders.pop_front();
            } else {
                // 3b. Resting order partially fills the incoming order
                front_order.fill(remaining_quantity);
                Trade new_trade = create_trade(order_id, front_order.get_order_id(), broker_id,
                                               front_order.get_trader_id(),
```

```
                                                    ticker,
                                                front_order.get_price(),
                                                    remaining_quantity,
                                                    side);
                trade_publisher->try_publish(new_trade);
                trade_events.emplace(std::move(new_trade));
                remaining_quantity = 0;
            }
        }

        // 4. Remove the price level if all resting orders are exhausted
        if (best_level_orders.empty()) {
            far_side.erase(best_level_price);
        }
    }

    // 5. If the incoming limit order is not fully filled, add it to the resting book (Maker)
    if (remaining_quantity > 0 && price != MARKET_BID_ORDER_PRICE &&
        price != MARKET_ASK_ORDER_PRICE) {
        order_id_map[order_id] = near_side[price].emplace(near_side[price].end(), order_id, price,
    }
}
```

Listing 1: Limit Order Book Matching Algorithm

2.2.5 Distribution Layer: Market Data Processor

The Market Data Processor (MDP) is implemented as an independent C++ application running a continuous event loop. It instantiates dedicated shared-memory ring buffer consumers for each active trading symbol to process both trades and order book snapshots. During each iteration of its busy-wait loop, the MDP invokes non-blocking `try_pop()` operations on these queues to check for new events. When raw trade or order book data structures are detected, the processor utilizes the `nlohmann::json` library to serialize the binary C++ objects into JSON payloads. Finally, the serialized data is pushed asynchronously to all active subscribers via the WebSocket server, delivering tick-by-tick market updates to the frontend dashboard.

2.2.6 Simulation Layer: Agents & Market Regimes

The simulation layer was constructed utilizing a set of C++ autonomous agent classes (`NoiseTrader`, `MarketMaker`, and `InformedTrader`) that act as independent clients to the exchange. They interface with the system using the same `FixClient` SDK as human users. Agent actions are driven by background threads, with the `NoiseTrader` specifically utilizing standard library random number generators (`std::poisson_distribution` and `std::mt19937`) to model the stochastic arrival of retail orders. Meanwhile, the `InformedTrader` leverages a specialized `OracleService` process that simulates a fundamental asset price

track. The oracle disseminates this continuous price path by broadcasting updates over a dedicated WebSocket channel, which the informed agent consumes to continuously poll for price deviations against the live order book and execute its sniping strategy.

Listing 2 illustrates the core logic for the `MarketMaker` agent based on the Avellaneda-Stoikov model. The agent dynamically adjusts its reservation price and optimal spread based on real-time market variance and its current inventory risk.

```
void MarketMaker::make_market(const std::string& ticker) {
    OrderBookSnapshot ob = m_md_handler->get_latest_orderbook(ticker);
    double mid_price = ob.mid_price() == 0.0 ? 100.0 : ob.mid_price();
    double variance = m_md_handler->calculate_variance(ticker);
    variance = variance == 0.0 ? 0.01 : variance;

    double q = m_fix_client->get_inventory(ticker);
    double q_centered = q - m_initial_inventory;

    auto now = std::chrono::steady_clock::now();
    std::chrono::duration<double> elapsed = now - m_start_time;
    double time_left = std::max(0.0, (m_terminal_time - elapsed.count()
        ) / m_terminal_time);

    // Calculate Reservation Price (r)
    double reservation_price = mid_price - (q_centered * m_gamma *
        variance * time_left);

    // Calculate Optimal Spread
    double spread = (m_gamma * variance * time_left) +
                    ((2.0 / m_gamma) * std::log(1.0 + (m_gamma / m_k)))
                        ;

    double optimal_bid = reservation_price - (spread / 2.0);
    double optimal_ask = reservation_price + (spread / 2.0);

    place_quotes(ticker, optimal_bid, optimal_ask, spread);
}
```

Listing 2: Avellaneda-Stoikov Market Maker Strategy

2.2.7 Server Setup and Deployment

To orchestrate the system across a Linux environment, we developed a custom Python-based deployment script. This application functions as a localized control plane that parses TOML configuration files to manage the lifecycle of the various microservices. It automatically validates service names, resolves listening ports, and ensures services are fully ready or safely terminated. By handling the automated startup and teardown of the components—including the FIX Gateway, Order Manager, MatchingEngine, and Market Data Processor—the deployment server provides a reliable and reproducible environment for running extensive market simulations.

2.2.8 Software Engineering Practices

Throughout the development of this complex system, we adhered to modern C++ software engineering principles and best practices to ensure robustness, maintainability, and performance:

- **Modern C++ Features:** We extensively utilized features from modern C++ standards (C++20/23) to write safer and more expressive code:
  - `std::expected`**:** Instead of relying heavily on exceptions, which can introduce non-deterministic control flow and performance overhead on the critical path, we used `std::expected` to return values or error states explicitly. This functional approach makes error handling predictable and type-safe.
  - **Monadic Operations (`std::transform` / `or_else`):** We leveraged monadic operations on `std::expected` and `std::optional` to chain operations cleanly, avoiding deep nesting of error-checking code and improving readability across our network and service layers.
  - **Smart Pointers and RAII:** Resource Acquisition Is Initialization (RAII) and smart pointers (`std::unique_ptr`, `std::shared_ptr`) were systematically applied across all components. This guaranteed deterministic resource management and strict prevention of memory leaks without relying on manual memory deallocation.
- **Design by Contract & Assertions:** To catch logic errors early, we adopted the "Design by Contract" methodology using the `boost::contract` library. We liberally applied preconditions, postconditions, and invariant assertions across critical components like the Limit Order Book and Balance Checker. This strict enforcement guarantees that functions only operate when their required state is valid, immediately asserting and failing fast if an invariant (such as an invalid order ID or negative balance) is violated.
- **Design Patterns & Abstractions:** We employed various structural and creational design patterns to decouple system components. The Factory Pattern (e.g., `MatchingEngineDependencyFactory`) was utilized to abstract the instantiation of complex dependencies in the MatchingEngine and Order Manager. Furthermore, abstract interfaces (e.g., `InboundServer`, `Publisher`) and dependency injection were used throughout the architecture. This allowed us to easily swap out implementations (for instance, substituting a real WebSocket server with a mock server during unit testing) without modifying the core business logic.
- **Asynchronous Logging (`spdlog`):** For observability, we integrated the high-

performance `spdlog` library. To prevent I/O operations from blocking the main execution threads, we configured it with asynchronous sinks (`spdlog::async_factory`). The logger also supports configurable log levels via environment variables and includes comprehensive signal handlers (`SIGINT`, `SIGTERM`) to ensure all pending logs are safely flushed to disk before a graceful shutdown.

2.3 Testing

2.3.1 Test Plan and Methodology

Our testing strategy encompassed three primary phases: unit testing, integration testing, and performance benchmarking. We initially utilized the Catch2 library for our unit testing framework due to its lightweight and expressive nature. However, as the system's complexity grew, we migrated our test suite to Google Test (gtest). This transition was motivated by Google Test's robust mock-testing facilities (via Google Mock), which allowed us to effectively isolate components using dependency injection.

For unit testing, we verified the core components—including the Client SDK, FIX Gateway, Order Manager, MatchingEngine, and Market Data Processor—by simulating diverse order scenarios. A key focus was validating the Price-Time Priority algorithm to ensure orders were filled in the correct sequence. We also performed "death testing" using Google Test's `EXPECT_DEATH` macro (as seen in our `limit_order_book_test.cpp`) to verify that the system correctly terminates or asserts when encountering unrecoverable states, such as querying invalid order IDs.

Integration testing focused on the data flow from the Client SDK through the FIX Gateway and Order Manager to ensure that Protobuf serialization and WebSocket communication maintained data integrity. We tested the trade events emitted by the Matching Engine to ensure the end-to-end flow functioned correctly. Regarding stress testing, our high-load evaluations were scoped strictly to individual component benchmarking, such as isolating the MatchingEngine's throughput and latency under massive order bursts.

A comprehensive execution matrix of the system's test suite, detailing critical scenarios and expected outcomes across multiple microservices, can be found in Appendix 6.4.

2.3.2 Locating Logic Errors and Bugs

Our testing procedures were critical in identifying flaws that were not initially apparent. A significant logic error was discovered regarding a Limit Order Book (LOB) invariant violation. During early testing, the MatchingEngine incorrectly attempted to match against the worst ask price rather than the best ask price, as the engine was accessing the end of the ascending price list instead of the beginning. While simple unit tests missed

this edge case, running a randomized order submission simulation revealed that "Ask" prices were crossing below "Bid" prices—a clear violation of market invariants.

This discovery prompted us to adopt the Design by Contract principle, writing more detailed test cases and enforcing preconditions, postconditions, and invariant assertions throughout the MatchingEngine code to catch logic bugs early.

2.4 Evaluation

2.4.1 Benchmarking

We used Google Benchmark across four Git branches. All benchmarks were executed on an AMD Ryzen AI 7 PRO 360 machine with 32 GB RAM running Linux, compiled with GCC utilizing `-O3` optimization flags. The baseline is the `benchmark` branch, built in Release mode with assertions and contract checks disabled. The `lob-dsa`, `mem-pool`, and `template` branches are exploratory prototypes for potential future improvements, rather than immediate production defaults.

MatchingEngine

Figure 6 compares the 10,000-order workload across the main latency-sensitive operations. The `memory-pool` branch is the only variant that consistently improves end-to-end performance over the baseline. Its strongest gain is in no-match order insertion (`BM_MatchingEngine_NewOrderNoMatchBurstLatency`), where latency is reduced from 0.807 ms to 0.434 ms ($\sim$1.86$\times$ faster). It also reduces pure LOB insertion latency from 0.672 ms to 0.257 ms ($\sim$2.61$\times$ faster). This significant speedup occurs because a custom memory pool pre-allocates contiguous blocks of memory at startup, completely bypassing the heavy operating system overhead of dynamic heap allocations (e.g., `malloc` or `new`) during rapid bursts of order insertions. In contrast, the `template-specialization` branch is mostly neutral to slightly slower, and the `lob-dsa` branch is effectively neutral for these engine-level benchmarks.

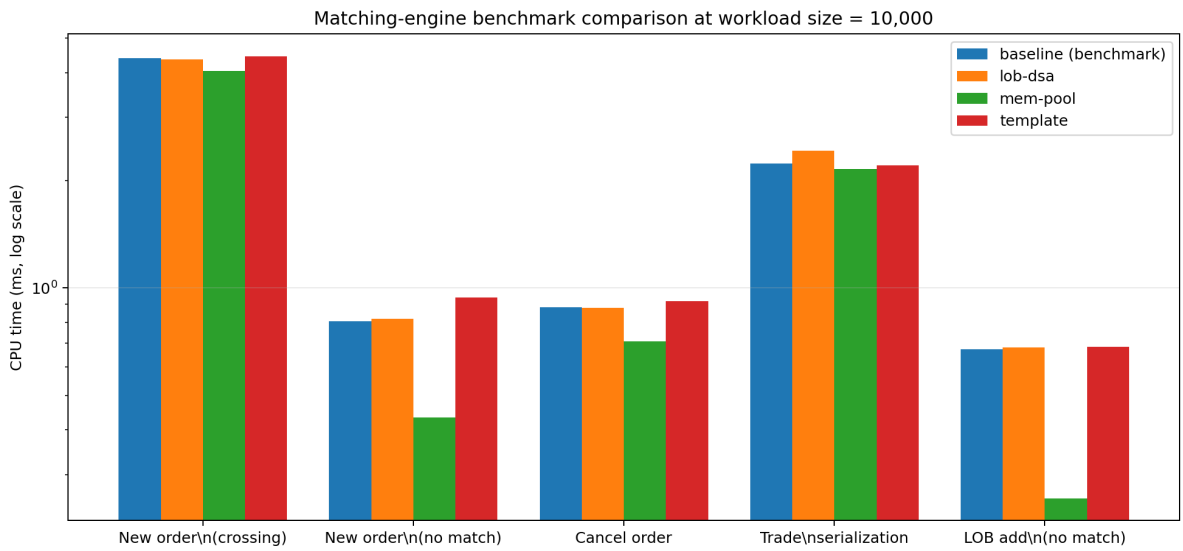


Figure 6: Matching-engine latency comparison across optimization branches at workload size 10,000 (log-scale y-axis).

To isolate the data-structure effect explored in the `lob-dsa` branch, we also benchmarked a realistic mixed-flow LOB scenario up to 1,000,000 orders (Figure 7). The vector-based level layout improves large-scale throughput versus the map/list baseline (631.11 ms $\rightarrow$ 353.48 ms at 1,000,000 orders, $\sim$1.79$\times$ faster), while the flat-vector variant degrades sharply at scale due to expensive shifting and maintenance costs under heavy mixed updates. Specifically, inserting or removing an order in the middle of a massive, contiguous flat-vector requires shifting all subsequent elements, meaning the $\mathcal{O}(N)$ algorithmic cost eventually outweighs any hardware benefits gained from CPU cache locality. This result indicates that contiguous storage can outperform node-based structures when the access pattern remains locality-friendly, but only when insertion/cancellation costs are controlled.

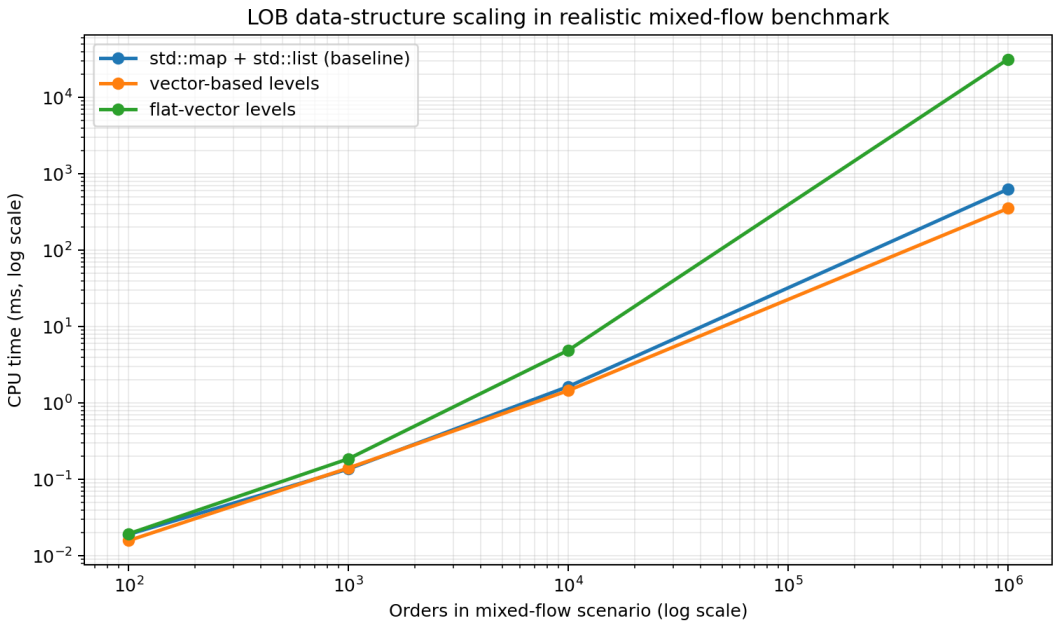


Figure 7: LOB data-structure scaling in realistic mixed-flow benchmarks on the `lob-dsa` branch (log-scale axes).

Table 3 reports baseline matching-engine throughput for core 10,000-order workloads. Depending on operation type, throughput ranges from 2.28 million to 12.40 million operations per second.

Benchmark (10,000 workload)	**Latency (ms)**	**Throughput (ops/s)**
New order (crossing, with matching)	4.391	2,277,274
New order (no match)	0.807	12,395,537
Cancel order	0.882	11,333,297
Trade serialization	2.233	4,478,941

Table 3: Baseline matching-engine throughput on the `benchmark` branch.

These exploratory benchmarks successfully identified improvements to our initial design specification: moving from standard dynamic memory allocation to a custom memory pool is necessary for production-grade throughput, and node-based structures can be effectively replaced by contiguous arrays when carefully managing insertion and deletion costs.

MPSC Lock-Free Ring Buffer

For the ring buffer component, we measured single-event producer-consumer handoff latency. On the `benchmark` branch rerun, we observed 771 ns in a single run, with aggregate

mean 648.76 ns (manual-time) and 673 ns (CPU-time). This benchmark currently has one configured input amount (one event handoff), shown in Figure 8.

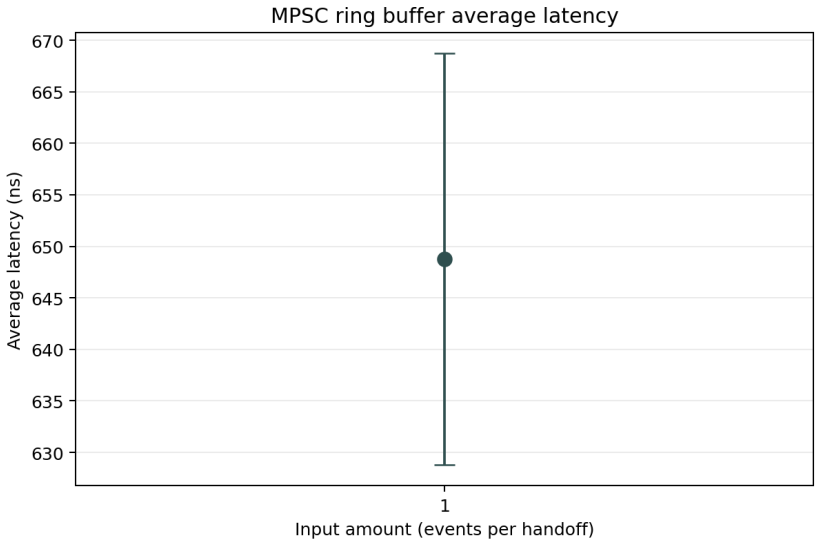


Figure 8: MPSC ring buffer average latency by configured input amount.

WebSocket

For the websocket benchmark on the baseline branch, we measured mean round-trip latency at three payload sizes:

- 64 B: 0.03698 ms (∼27,042 round-trips/s)
- 512 B: 0.04106 ms (∼24,353 round-trips/s)
- 4096 B: 0.06227 ms (∼16,059 round-trips/s)

This shows clear latency growth as payload size increases in our local setup.

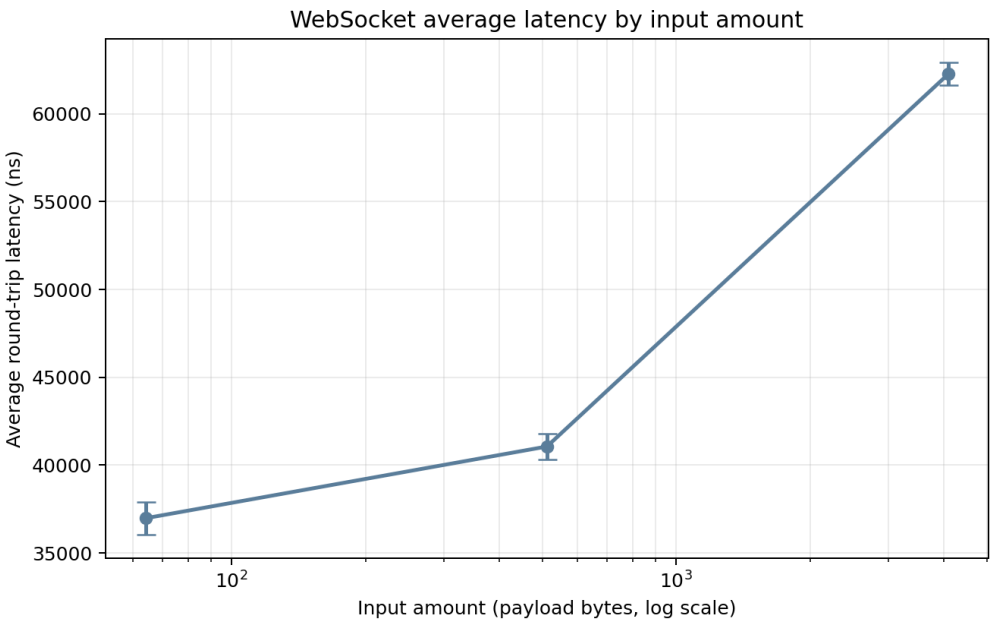


Figure 9: WebSocket average round-trip latency by input amount.

2.4.2 Simulation Realism

Discuss stylized facts here.

3 Discussion

This project successfully delivered a highly modular and functional simulated exchange environment, but it also served as a profound learning experience regarding the extreme engineering constraints of algorithmic trading systems. In this section, we evaluate the deeper reasons behind our successes and shortcomings, and discuss the architectural limitations that would prevent this system from being deployed in a live, high-frequency trading (HFT) production environment.

3.1 Project Evaluation and Reflections

We are most confident in our achievement of building a low-latency, strictly correct Limit Order Book and an intuitive C++ SDK. Our success here stems directly from our rigorous adoption of modern C++ paradigms, strict memory management (RAII), and our choice of lock-free data structures. By explicitly isolating the matching engine from network and database I/O, we ensured that the core matching logic—the most critical component of the exchange—could perform optimally without being bottlenecked by external dependencies.

Our approach to the system's distributed architecture also went exceptionally well in terms of logical decoupling. Separating concerns across the FIX Gateway, Order Manager, MatchingEngine, and Market Data Processor allowed us to build an elegant "game server" provisioning model.

However, our success with realistic market simulation was only partial. While we successfully implemented the mathematical foundations of the Avellaneda-Stoikov market maker and Geometric Brownian Motion, we discovered that generating truly realistic macro-market behaviour requires vastly more sophisticated parameter calibration and multi-agent interaction than a single project timeframe permits. The failure to perfectly replicate real-world volatility was not a flaw in the code, but rather an underestimation of the sheer mathematical complexity involved in modelling human psychological trading factors and macro-economic events.

3.2 System Limitations

While our platform serves perfectly as an educational sandbox, several critical factors and architectural shortcuts would discourage its use in a real-world financial institution:

- **Network Latency and IPC Overhead:** We relied heavily on WebSockets for inter-service communication. Even when services are deployed on the same machine, the encapsulation and de-encapsulation overhead of WebSockets is computationally

intensive, preventing nanosecond-scale determinism.

- **Cloud Deployment and Load Balancing:** Currently, our deployment orchestration utilizes a Python script locally. We did not leverage separate AWS EC2 instances to test true cross-host microservice deployment or hardware load balancing.
- **Auto-Recovery and Robustness:** The system currently lacks comprehensive auto-recovery mechanisms. In a real exchange, if the Order Manager or Matching Engine crashes, it currently results in dropped messages rather than seamlessly failing over to a hot-standby replica.
- **Concurrency in the MatchingEngine:** Our MatchingEngine currently processes orders sequentially, which limits the maximum theoretical throughput under massive concurrent load. In industry, this is typically solved by partitioning individual Limit Order Books across independent CPU cores and utilizing the LMAX Disruptor pattern or deterministic state machine replication to achieve massive, lock-free concurrent execution.
- **Database Scalability:** While QuestDB handles our current tick data effectively, scaling to thousands of concurrent users would eventually bottleneck our single-node storage layer due to heavy disk I/O load.
- **Rate Limiting and Flow Control:** The system currently lacks strict rate-limiting at the gateway layer. Without throttling mechanisms, sudden bursts of millions of orders could easily cause buffer overflows or memory exhaustion across the microservice pipeline.

4 Conclusions

This project successfully culminated in the development of a highly modular, low-latency simulated stock exchange designed for algorithmic trading education. Our major technical accomplishments include the implementation of a strictly correct, lock-free Limit Order Book, an intuitive C++ Trading SDK, a dual-database persistence layer, and a modern React-based frontend that orchestrates isolated trading "game servers." Throughout this process, we gained profound insights into the stringent performance and reliability constraints of financial microservices, particularly regarding modern C++ memory management, IPC shared memory, and event-driven architectures.

While we successfully built the core trading pipeline, several problems remain unaddressed for a production-ready system, highlighting exciting avenues for future work. To remedy the system's current limitations, we recommend the following enhancements:

- Future iterations should replace WebSockets for internal communication with ultra-low-latency alternatives like Unix domain pipes, UDP multicast, or hardware-accelerated kernel bypassing techniques (e.g., `AF_XDP` or DPDK) to eliminate operating system networking stack overhead.
- Transitioning from local Python deployment to true CI/CD pipelines deploying across distributed AWS EC2 instances. This should be paired with hardware load balancers and auto-recovery mechanisms (such as distributed consensus and persistent event sourcing) to ensure robust failover.
- The matching engine's throughput can be drastically improved by assigning independent CPU threads to different tickers, yielding near-linear scaling without lock contention. Additionally, database sharding should be implemented to partition user and tick data across multiple nodes.
- Incorporating more sophisticated mathematical models or AI-driven agents could improve market realism. Furthermore, conducting extensive user testing with volunteer algorithmic traders would provide better data on the SDK's usability and system stability under unpredictable, malicious, or massive loads (e.g., implementing and testing rate-limiting).

5 References

6 Appendix

6.1 Explanation on Limit Order Books

Electronic markets are governed by centralized Limit Order Books (LOBs), where heterogeneous agents submit, cancel, and execute orders that collectively form price, liquidity, and volatility dynamics. Each Limit Order Book contains a buy side (also called bids) and a sell side (also called asks). If the bid price is greater than or equal to the ask price, the quantity matched will be removed from the LOB. Orders are first sorted by price, then sorted by the earliest time of arrival. This is known as *price-time priority*.

For example, suppose we start with an empty LOB. A trader submits a buy order for one share of APPL at $100. Since there is no one selling the desired share, this order sits at the "top" of the LOB as the best bid. When another trader submits a buy order of one share of APPL at $99, this bid sits *behind* the $100 bid due to price-time priority. If another trader then submits a sell order for one APPL share at $100, there is now a *match* between bid and ask prices, and those orders are filled. All that remains on the LOB is the bid at $99, becoming the new best bid. If a seller places an ask for $105, it is clear that no matching is available. This is known as a *bid-ask spread* of $5. The above example describes limit orders. Another popular type of orders are market orders, which match with the best prices immediately.

This simple mechanism allows for market participants to provide and extract information regarding an asset. For example, a larger bid-ask spread is correlated with lower liquidity, reflecting a riskier trade when entering a position. By aggregating buyer and seller intentions in a central LOB, traders are able to infer the fair price of an asset.

6.2 Project Planning

6.2.1 Distribution of Work & Gantt Chart

G - Gordon J - Josh O - Om										
Task	**PIC**	**Aug**	**Sep**	**Oct**	**Nov**	**Dec**	**Jan**	**Feb**	**Mar**	**Apr**
Perform Literature Survey	O									
Write Proposal Report	G									
Design & Implement Limit Order Book	J									
Design & Implement Gateway	O									
Design & Implement Order Manager	G									
Design & Implement Market Data Processor	J									
Design & Implement Trade Interface and SDK	J									
Test Limit Order Book	G									
Test Gateway	O									
Test Order Manager	G									
Test Market Data Processor	J									
Test Trade Interface and SDK	J									
Design & Implement System Architecture & Infrastructure	G									
Design & Implement Traffic & Connection Management	O									
Test System Architecture & Infrastructure	G									
Test Traffic & Connection Management	O									
Design, Implement & Test Market Simulation Models	O									
Write Progress Report	J									
Write Final Report	G									
Record Presentation Video	J									
Prepare Presentation Deck	O									

Table 4: Project Timeline and Task Distribution

6.3 Required hardware & software

6.3.1 Hardware

- Linux x86_64 machine (Fedora 43, kernel 6.19.8)
- CPU: AMD Ryzen 5 6600H (6 cores / 12 threads, up to 4.57 GHz)
- Memory: 32 GiB RAM

6.3.2 Software

- React and JavaScript for frontend development
- C++23 as core programming language
- PostgreSQL (locally hosted)
- QuestDB (locally hosted)

6.4 Comprehensive Test Cases

Table 5 provides a comprehensive matrix of test cases extracted from the Google Test suites across multiple backend services (MatchingEngine, Order Manager, and Backend Service) and Catch2 suites for common libraries. While we initially used Catch2 for its lightweight nature and continued to use it for our base libraries, we migrated to Google Test for our complex backend services to leverage its robust mocking framework. This matrix serves as evidence of the rigorous execution testing and validation of system invariants.

Table 5: Comprehensive execution matrix of test cases across backend services and common libraries.

Test Name	**Component**	**Scenario/ Description**	**Expected Result**	**Actual Result**	**Pass /Fail**
`MatchMultiple StandingBid Orders`	Matching Engine	Aggressive ask order matches against multiple limit buy orders at the same price level.	Orders matched strictly in arrival time sequence (Price-Time Priority).	Matches generated in correct temporal order.	Pass
`CompletelyFill StandingBid Order`	Matching Engine	Aggressive ask order crosses and exactly fills resting bid order.	Single trade executed; resting bid completely removed from LOB.	Trade generated; LOB updated correctly.	Pass
`PartiallyFill MarketAsk`	Matching Engine	Market ask order partially filled by available resting bids.	Trade generated for available quantity; market order fully consumed.	Trade generated; remainder of book unmodified.	Pass

Continued on next page

Table 5: Comprehensive execution matrix of test cases across backend services and common libraries (continued)

Test Name	**Component**	**Scenario/ Description**	**Expected Result**	**Actual Result**	**Pass /Fail**
`AddMarketOrder ToEmptyBook`	Matching Engine	Market order submitted when no opposing liquidity exists.	Order rejected or stored temporarily (depending on rule) with zero fill.	Handled gracefully without crash.	Pass
`CancelInvalid Order`	Matching Engine	`CancelOrderDeath Test` attempts to cancel a non-existent Order ID.	Contract assertion failure / death test triggered gracefully.	Process terminated via assertion as expected.	Pass
`GetInvalid OrderID`	Matching Engine	`GettersDeathTest` queries a non-existent Order ID.	Contract assertion failure.	Process terminated via assertion.	Pass
`Insufficient Balance`	Order Manager	`HasSufficient BalanceTest` checks if a trade exceeds current broker balance.	Validator returns false; order rejected before routing.	Returned false; balance remains unchanged.	Pass
`DecrementBelow Zero`	Order Manager	`UpdateBalance DeathTest` attempts to decrement balance below 0.	Contract assertion failure / death test triggered.	Process terminated via assertion.	Pass
`InvalidCancel Request`	Order Manager	Validation of an improperly formed cancel request payload.	Validator flags payload as malformed.	Rejected prior to matching engine.	Pass

Continued on next page

Table 5: Comprehensive execution matrix of test cases across backend services and common libraries (continued)

Test Name	**Component**	**Scenario/ Description**	**Expected Result**	**Actual Result**	**Pass /Fail**
`ValidMarketBid`	Order Manager	Validation of a properly formed market bid payload.	Passes semantic checks; forwards to engine.	Successfully forwarded to Matching Engine.	Pass
`ForwardAnd ReplyFails`	Order Manager	`ForwardAndReply Test` checks behavior when MatchingEngine connection drops.	Order Manager gracefully handles socket failure and replies with rejection.	Rejection report sent to user.	Pass
`LoginWrong Password`	Backend Service	User attempts login with incorrect credentials.	Service returns 401 Unautho-rized.	HTTP 401 returned.	Pass
`Signup DuplicateUser`	Backend Service	Registration attempt with an existing email/username.	Service returns 409 Conflict or 400 Bad Request.	Correct error code returned.	Pass
`BasicEnqueue Dequeue`	Thread Safe Queue (Catch2)	Single-threaded enqueue and dequeue operations.	Elements are enqueued and dequeued in FIFO order.	Operations succeed and order is preserved.	Pass
`Multithreaded Producer Consumer Correctness`	Thread Safe Queue (Catch2)	Multiple producers and consumers operating concurrently.	All elements are processed without data races or deadlocks.	Data integrity maintained across threads.	Pass

Continued on next page

Table 5: Comprehensive execution matrix of test cases across backend services and common libraries (continued)

Test Name	**Component**	**Scenario/ Description**	**Expected Result**	**Actual Result**	**Pass /Fail**
`insert_ order succeeds and row is queryable`	Database Client (Catch2)	Inserting a new order record into the database.	Order is successfully inserted and can be retrieved.	Record stored and queried successfully.	Pass
`concurrent insert_ order from multiple clients`	Database Client (Catch2)	High-concurrency insertions from multiple threads.	All orders are inserted accurately without dropped transactions.	Database handles concurrent writes correctly.	Pass
`MPSC Producer/ Consumer`	Lock-free Queue (Catch2)	Single producer and consumer operating over a lock-free queue.	Messages are safely passed without locks.	High-throughput message passing achieved.	Pass
`Client Server Ping Pong`	Network/ WebSocket (Catch2)	Bi-directional message echoing between client and server.	Messages are received intact and in order.	Successful real-time communica-tion.	Pass
`Two Client One Server`	Network/ WebSocket (Catch2)	Multiple clients connecting and communicating with a single server concurrently.	Server correctly identifies and routes messages to respective clients.	Multiplexed communica-tion successful.	Pass